



CourseCube®
(Formerly Java Learning Center)

MicroServices

Study Guide

Module 1

Introduction to MicroServices

Author
Srinivas Dande





CourseCube[®]
(Formerly Java Learning Center)



Introduction to MicroServices

A. Monolithic Architecture :

- Monolithic Architecture is a traditional architecture for enterprise applications.
- Dependencies are managed by dividing the application into layers that perform logical functions as follows
 - **Presentation** — responsible for handling HTTP requests and responding with either HTML or JSON/XML (for web services APIs).
 - **Business logic** — the application's business logic.
 - **Database access** — data access objects responsible for access the database.
 - **Application integration** — integration with other services (e.g. via messaging or REST API).
- A layer can only call into layers that sit next to it. It can be hard to introduce changes in one part of the application without touching the rest of the application. That makes complex for frequent updates and limiting how quickly new features can be added.
- A monolithic application has single code base with multiple modules. Modules are divided as either for business features. It has single build system which builds entire application. It also has single executable or deployable binary.

a) Benefits of Monolithic Architecture:

- Simple to develop.
- Simple to test. For example you can implement end-to-end testing by simply launching the application and testing the UI with Selenium.
- Simple to deploy. You just have to copy the packaged application to a server.
- Simple to scale horizontally by running multiple copies behind a load balancer.
- In the early stages of the project it works well and basically most of the big and successful applications which exist today were started as a monolith.
- Prefer for Traditional business domains which has low Frequency of updates



b) Challenges with Monolithic Applications:

- 1) Large monolithic code base makes complicated to understand, especially for new developer.
 - 2) Overloaded IDE, Large code base makes IDE slow, build time increase.
 - 3) Deployment challenges.
Any change in the any one module would require the entire application to be deployed again.
Continuous deployment is difficult.
You must redeploy the entire application on each update.
 - 4) Scaling become challenging.
 - 5) Big database.
 - 6) Fault Tolerance.
If a specific feature is not working it will make the system goes down. In order to handle this problem, the system has to be rebuilt, retested and redeployed

Bug in any module (e.g. memory leak) can potentially bring down the entire process. Moreover, since all instances of the application are identical, that bug will impact the availability of the entire application.
 - 7) Monolithic applications have a problem to adopting new technologies. Since changes in frameworks or languages will affect an entire application it is extremely expensive in both time and cost.
 - 8) Extremely difficult to change technology or language or framework (hybrid technologies) because everything is tightly coupled and depend up on each other.
- Application architecture has always been evolving alongside demanding business requirements and the evolution of technologies.

B. What are microservices?

- 1) Microservices are small, independent, and loosely coupled.
- 2) A single small team of developers can write and maintain a service.
- 3) Each service has a separate codebase, which can be managed by a small development team.
- 4) Services can be deployed independently. A team can update an existing service without rebuilding and redeploying the entire application.
- 5) Services are responsible for persisting their own data or external state. This differs from the traditional model, where a separate data layer handles data persistence.
- 6) Services communicate with each other by using well-defined APIs. Internal implementation details of each service are hidden from other services.
- 7) Services don't need to share the same technology stack, libraries, or frameworks.
- 8) **Prefer for Complicated domain which has high Frequency of updates.**

a) Microservices benefits

1) Supports polyglot architecture

Each Microservice can run with its own architecture or technology or different versions of same technology.

2) Enabling experimentation and innovation

With large monolithic applications, experimentation was not easy. With Microservices, it is possible to write a small microservice to achieve the targeted functionality and plug it into the system.

3) Small, focused teams:

A microservice should be small enough that a Small team can build, test, and deploy it. Small team sizes promote greater agility. Large teams tend to be less productive, because communication is slower, management overhead goes up, and agility diminishes.



4) Agility:

Because microservices are deployed independently, it's easier to manage bug fixes and feature releases. You can update a service without redeploying the entire application, and roll back an update if something goes wrong. In many traditional applications, if a bug is found in one part of the application, it can block the entire release process. New features may be held up waiting for a bug fix to be integrated, tested, and published.

5) Small code base:

In a monolithic application, there is a tendency over time for code dependencies. Adding a new feature requires touching code in a lot of places. By not sharing code or data stores, a Microservices architecture minimizes dependencies, and that makes it easier to add new features.

6) Selective scaling

A monolithic application, packaged as a single WAR or an EAR, can only be scaled as a whole. In Microservices, each service could be independently scaled up or down depending on scalability requirement. As scalability can be selectively applied at each service, the cost of scaling is comparatively less with the microservices approach.

7) Fault isolation:

If an individual microservice becomes unavailable, it won't disrupt the entire application, as long as any upstream microservices are designed to handle faults correctly.

8) Data isolation:

It is much easier to perform schema updates, because only a single microservice is affected. In a monolithic application, schema updates can become very challenging, because different parts of the application may all touch the same data, making any alterations to the schema risky.

9) Allowing substitution

Microservices are self-contained, independent deployment modules enabling the substitution of one microservice with another similar microservice. Many large enterprises follow buy-versus-build policies to implement software systems. A common scenario is to build most of the functions in house and buy certain niche capabilities from specialists outside.

10) **Supporting Cloud**

11) **Enabling DevOps**

Microservices are one of the key enablers of DevOps. DevOps is widely adopted as a practice in many enterprises, primarily to increase the speed of delivery and agility. DevOps advocates having agile development, high-velocity release cycles, automatic testing, and automated deployment.

b) Challenges with MicroService Architecture

- The benefits of microservices don't come for free. Here are some of the challenges to consider before embarking on a microservices architecture.

1) Complexity:

A microservices application has more moving parts than the equivalent monolithic application. Each service is simpler, but the entire system as a whole is more complex.

2) Development and testing:

Existing tools are not always designed to work with service dependencies. Refactoring across service boundaries can be difficult. It is also challenging to test service dependencies, especially when the application is evolving quickly.

3) Lack of governance:

The decentralized approach to building microservices has advantages, but it can also lead to problems. You may end up with so many different languages and frameworks that the application becomes hard to maintain. It may be useful to put some project-wide standards in place, without overly restricting teams' flexibility. This especially applies to cross-cutting functionality such as logging.

4) Network congestion and latency:

The use of many small, granular services can result in more interservice communication. Also, if the chain of service dependencies gets too long (service A calls B, which calls C...), the additional latency can become a problem. You will need to design APIs carefully. Avoid overly chatty APIs, think about serialization formats, and look for places to use asynchronous communication patterns.

5) Data integrity :

With each microservice responsible for its own data persistence. As a result, data consistency can be a challenge. Embrace eventual consistency where possible.

6) Management:

To be successful with microservices requires a mature DevOps culture. Correlated logging across services can be challenging. Typically, logging must correlate multiple service calls for a single user operation.

7) Skillset:

Microservices are highly distributed systems. Carefully evaluate whether the team has the skills and experience to be successful.

c) Key Points about MicroService Architecture

1) In the microservices architecture, there is no more application development rather service development.

2) Microservices expose service endpoints as APIs and abstract all their realization details i.e., the internal implementation logic, architecture, and technologies are completely hidden behind the service API.

3) Messaging (JMS / AMQP), HTTP, and REST are commonly used for interaction means communication between microservices.

4) Microservices are reusable business services.

5) Well-designed microservices are stateless and does not share any conversational state maintained by the services.

6) Microservices are discoverable.

d) Number Of Endpoints for a Microservice

- The number of endpoints is not really a decision point. In some cases, there may be only one endpoint, whereas in some other cases, there can be more than one endpoint in a microservice.



e) Communication styles

- Communication between microservices can be happen in two ways.
 - Synchronous Communication
 - ASynchronous Communication
- **SYNchronous Communication**
 - In synchronous communication, the http listener such as tomcat or jetty or jboss, etc is needed but not messaging listener. When a caller requests a service, it passes the required information and waits for a response.

Advantages:

- 1) No messaging server overhead.
- 2) The error will be propagated back to the caller immediately.

Disadvantages:

- 1) The caller has to wait until the request has been processed.
- 2) Adds hard dependencies (tight coupling) between Microservices i.e., if one service in the chain fails, then the entire service chain will fail.

- **ASYNchronous Communication**
 - The asynchronous style is based on reactive event loop semantics which decouple microservices.

Advantages:

- 1) Decouple Microservices
- 2) Higher level of scalability because services are independent. Hence if there is a slowdown in one of the services, it will not impact the entire chain.

Disadvantages:

- 1) It has a dependency to an external messaging listener.
- 2) It is complex to handle the fault tolerance of a messaging listener.



f) Number Of VMs per MicroService

- The one microservice can be deployed in one or multiple Virtual Machines (VMs) by replicating the deployment for scalability and availability.
- Multiple Microservices can be deployed in one VM if the service is simple and the traffic volume is less.
- In case of cloud infrastructure, the developers need not to worry about where the services are running. Developers may not even think about capacity planning. Services will be deployed in a compute cloud. Based on the infrastructure availability and the nature of the service, the infrastructure self-manages deployments.

g) Can microservices share data stores?

- In principle, microservices should abstract presentation, business logic, and data stores i.e., each microservice logically could use an independent database.
- Shared data models, Shared schema, and shared tables are disasters when developing microservices.
- If the services have only a few tables, it may not be worth investing a full instance of a database like MySQL instance. In such cases, schema level segregation is good enough to start with.

h) Shared Libraries

- Sometimes code and libraries may be duplicated in order to adhere to autonomous and self-contained principle.

i) Microservices early adopters

Many organizations have already successfully embarked on their journey to the microservices world.

1) Netflix

Netflix, an international on-demand media streaming company, is a pioneer in the microservices space.

Netflix transformed their large pool of developers developing traditional monolithic code to smaller development teams producing microservices. At Netflix, engineers started with monolithic, went through the pain, and then broke the application into smaller units that are loosely coupled and aligned to the business capability.



2) Amazon

The well-architected monolithic application was based on a tiered architecture with many modular components. However, all these components were tightly coupled. As a result, Amazon was not able to speed up their development cycle. Amazon then separated out the code as independent functional services, wrapped with web services, and eventually advanced to microservices.

3) Twitter

When Twitter experienced growth in its user base, they went through an architecture-refactoring cycle. With this refactoring, Twitter moved away from a typical web application to an API-based event driven code. Twitter uses Scala and Java to develop microservices with polyglot persistence.

4) Uber

When Uber expanded their business from one city to multiple cities, the challenges started. Uber then moved to microservice based architecture by breaking the system into smaller independent units. Each module was given to different teams and empowered them to choose their language, framework, and database. Uber has many microservices deployed in their ecosystem using REST.

5) eBay :

In 2011, when eBay was introducing microservices, the company had 97 million active users and 62 billion gross merchandise volume. On a typical day, eBay IT systems had to deal with massive traffic, like 75 billion database calls, 4 billion page views and 250 billion search queries. The change to microservice architecture wasn't the first major switch of technology at eBay; the company faced similar transitions in 1999 and 2005.

Microservices were not invented; rather they were evolved from the previous architecture styles.



Building microservices with Spring Boot and Spring Cloud

- Traditionally a war was explicitly created and deployed on a Tomcat server. But microservices need to develop services as executables, self-contained JAR files with an embedded HTTP listener (such as tomcat or jetty).
- Spring boot is a tool to develop such kinds of services i.e., Spring Boot enables microservices development by packaging all the required runtime dependencies in a executable fat jar file.

Software Required

1. JDK 1.8
2. Spring Tool Suite
3. Maven 3.x /Gradle
4. Spring Framework 5.x RELEASE
5. Spring Boot 2.x RELEASE
6. RabbitMQ / Kafka
7. Git
8. Spring Cloud
9. MySQL DB

What You will learn Now

1. Cloud Config Server
 2. Cloud Config Client
 3. Cloud Bus
 4. Eureka
 5. Zuul
 6. Feign
 7. Ribbon
 8. Hysterix
 9. Sleuth
 10. RabbitMQ
- more



CourseCube[®]
(Formerly Java Learning Center)

JLC Bookstore x +

localhost:9000/showAllBooks?author=Srinivas&category=Spring

JLCBookStore Rate the books Show My Ratings Login Register

All Authors All Categories Show Books

Book ID	Book Name	Author	Publications	Category	Show My Cart
101	Master Spring Boot	Srinivas	JLC	Spring	Add to My Cart More Info
105	Master Spring Framework	Srinivas	JLC	Spring	Add to My Cart More Info
109	Master Spring Rest	Srinivas	JLC	Spring	Add to My Cart More Info

© 2020 JLCBookStore

Activate Windows
Go to Settings to activate Windows.

Windows Type here to search

JLC Bookstore x +

localhost:9000/showBookInfo?bookId=101

JLCBookStore Rate the books Show My Ratings Login Register

Book ID	101
Book Name	Master Spring Boot
Author	Srinivas
Publications	JLC
Category	Spring
Price	Rs. 2000.0
Offer	10% Off
Ratings	4.0

Continue Shopping

© 2020 JLCBookStore

Activate Windows
Go to Settings to activate Windows.

Windows Type here to search



CourseCube
(Formerly Java Learning Center)

JLC Bookstore x +

localhost:9000/showMyCart

JLCBookStore Rate the books Show My Ratings Login Register

Book ID	Book Name	Author	Publications	Category	Continue Shopping
101	Master Spring Boot	Srinivas	JLC	Spring	Remove from My Cart
105	Master Spring Framework	Srinivas	JLC	Spring	Remove from My Cart

Place My Order

© 2020 JLCBookStore

Activate Windows
Go to Settings to activate Windows.

Windows Taskbar: Type here to search, icons for Edge, File Explorer, Mail, Chrome, and others. System tray shows 20:36, 11-02-2020.

JLC Bookstore x +

localhost:9000/placeOrder

JLCBookStore Rate the books Show My Ratings Login Register

Your Order has been placed successfully

Continue Shopping

© 2020 JLCBookStore

Activate Windows
Go to Settings to activate Windows.

Windows Taskbar: Type here to search, icons for Edge, File Explorer, Mail, Chrome, and others. System tray shows 20:36, 11-02-2020.

JLC Bookstore x +

← → ↻ ⓘ localhost:9000/showRatingsForm 🔍 ☆ 🌐 🌐 🌐 m Paused ⋮

JLCBookStore [Rate the books](#) [Show My Ratings](#) [Login](#) [Register](#)

Provide Your Rating

User Id

Book Id

Rating

Review

[Add Rating Info](#)

© 2020 JLCBookStore

Activate Windows
Go to Settings to activate Windows.

Windows Taskbar: Type here to search, Edge, File Explorer, Store, Mail, Chrome, Task View, Settings, 20:37 11-02-2020, 4 notifications

JLC Bookstore x +

← → ↻ ⓘ localhost:9000/showMyRatings 🔍 ☆ 🌐 🌐 🌐 m Paused ⋮

JLCBookStore [Rate the books](#) [Show My Ratings](#) [Login](#) [Register](#)

Book ID	User ID	Rating	Review
101	U-12	4.5	Good
101	U-12	4.5	Excellent
101	U-12	3.0	Good

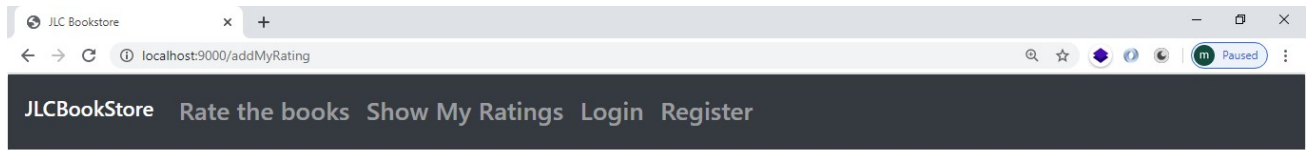
© 2020 JLCBookStore

Activate Windows
Go to Settings to activate Windows.

Windows Taskbar: Type here to search, Edge, File Explorer, Store, Mail, Chrome, Task View, Settings, 20:37 11-02-2020, 4 notifications



CourseCube®
(Formerly Java Learning Center)



Your Rating Added Successfully
100 Points added BookStore Wallet
Thank You

